System Verification and Validation Plan for Software Engineering

Team #14, Troitsky Toolbox
Pedro Azevedo
Mason Azzopardi
Michael Roberts
Jack Easton

November 3, 2026

Revision History

Date	Version	Notes
October 27, 2026	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **v**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
  - 2.2.1 Primary Objectives . . . 2
  - 2.2.2 Out-of-Scope Objectives . . . 2
  - 2.2.3 Assumptions . . . 3
- 2.3 Custom Documents . . . 3
  - 2.3.1 Fall - Interviews . . . 3
  - 2.3.2 Winter - Usability Report . . . 3
- 2.4 Relevant Documentation . . . 4

3 Plan **4**
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 6
- 3.3 Design Verification . . . 8
- 3.4 Verification and Validation Plan Verification . . . 8
- 3.5 Implementation Verification . . . 9
- 3.6 Automated Testing and Verification Tools . . . 10
- 3.7 Software Validation . . . 11

4 System Tests **12**
- 4.1 Tests for Functional Requirements . . . 12
  - 4.1.1 Web Server (S.2.1) . . . 13
  - 4.1.2 Access Control System (S.2.2) . . . 13
  - 4.1.3 Toolbox Broker (S.2.3) . . . 14
  - 4.1.4 Knowledge Repository (S.2.4) . . . 14
  - 4.1.5 Model Analysis System (S.2.5) . . . 15
  - 4.1.6 Performance Analysis System (S.2.5.1) . . . 16
  - 4.1.7 Rule Compliance System (S.2.5.2) . . . 17
  - 4.1.8 Material Testing Manager (S.2.6) . . . 17
  - 4.1.9 Knowledge Transfer Manager (S.2.7) . . . 18
- 4.2 Tests for Nonfunctional Requirements . . . 18
  - 4.2.1 Webserver . . . 19
  - 4.2.2 Access Control System . . . 21

4.2.3 Toolbox Broker . . . 22
4.2.4 Knowledge Repository . . . 24
4.2.5 Model Analysis System . . . 26
4.2.6 Performance Analysis System . . . 27
4.2.7 Rule Compliance System . . . 28
4.2.8 Material Testing Manager . . . 29
4.2.9 Knowledge Transfer Manager . . . 30
4.2.10 System-Wide Requirements . . . 31
4.3 Traceability Between Test Cases and Requirements . . . 33

5 Unit Test Description **36**
5.1 Unit Testing Scope . . . 36
5.2 Tests for Functional Requirements . . . 37
5.2.1 Bridge Failure Module . . . 38
5.2.2 Rules Compliance Module . . . 40
5.2.3 Bridge Scoring Module . . . 42
5.2.4 Log-in . . . 44
5.2.5 Materials Testing . . . 46
5.2.6 Knowledge Repository . . . 47
5.2.7 Knowledge Transfer . . . 49
5.2.8 Toolbox Broker . . . 50
5.3 Tests for Nonfunctional Requirements . . . 51
5.3.1 Module Analysis . . . 51
5.4 Traceability Between Test Cases and Modules . . . 52

6 Appendix **53**
6.1 Symbolic Parameters . . . 53
6.2 Usability Survey Questions? . . . 57

7 Appendix — Generative AI Use Disclosure **58**
7.1 AI Tools Used . . . 58
7.2 How and Where Used . . . 58
7.3 Verification of AI-Generated Content . . . 58

8 Appendix — Reflection **59**

List of Tables

1 Traceability between System Tests, SRS Requirements, and Hazard Analysis Requirements 34
2 Symbolic constants used by the system tests for functional requirements 53
3 Test data for the system tests for functional requirements . . . 55
4 Compliance models for RULESET_TEST. Their geometry is exact, so the extracted values equal those listed. 56

[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

The Troitsky Toolbox project seeks to create a centralized information repository and a collection of software services that provide technical support for the McMaster Troitsky club. These services ensure information transfer between years and teams, and facilitate support and onboarding for new members. Served as a web application, accessible by all current club members, the services provided will simplify the technical components of the bridge design process, decreasing the barrier to becoming a competitive team.

One of the primary services in the toolbox is the model analysis service, which allows teams to load STEP files of their bridge designs and perform basic analysis on the models. This service will provide teams with a simplified analysis of their design, when compared to external static model analysis software, and will integrate with the material testing component and the rule compliance checker to provide a more specified analysis given the build material properties and current competition rules. Services regarding the information repository will include the ability to search through past designs and load them into the model analysis service, allowing teams to upload their designs with exit documentation and providing customized material testing result upload features. A knowledge transfer service collects executive handoff documents so that club knowledge carries over between years. In managing the toolbox access, role-based access control (RBAC) will be integrated to ensure that information is kept within the club, limiting access, and allowing executive members the ability to manage club users.

2.2 Objectives

The primary objective of the verification and validation plan is to outline the approach to ensure that the Troitsky Toolbox project will meet the requirements outlined in the software requirements specification. This plan will help to build confidence in the correctness, reliability and safety of the system, focusing verification on high-risk points and safety-critical concerns outlined

in the hazard analysis.

The V&V plan will primarily focus on these system objectives outlined below. There are also goals outlined that have been deemed out of scope for the current development cycle, but will be outlined in documentation as goals for future year development.

2.2.1 Primary Objectives

- **Adequate model analysis correctness:** Testing the correctness of the model analysis service is vital to the usability of the toolbox, as, without ensured correctness, teams will not be able to rely on the results. These results will not only be used by teams in the bridge design process, but will also guide them in making estimates of their bridge's weight and ultimate load capacity.

- **Data integrity and persistence:** The system must ensure data integrity and security in the knowledge repository. The repository will serve as the club's main data storage and information transfer, so losing data would be a critical concern and would defeat the project's purpose.

- **Access control:** The RBAC in the club will maintain data privacy both between teams and outside of the club. Testing this access control will provide confidence to teams when storing current bridge designs in the system, and ensure that executive and exit documentation is private to the club.

- **Demonstrating functional usability:** The functional usability of the system determines if it will be fully integrated into the club's organization and individual McMaster teams' bridge design and development process.

2.2.2 Out-of-Scope Objectives

- **Industry standard static model analysis performance:** The model analysis service is intentionally designed to provide a simplified approximation of static model analysis, so its accuracy will have a larger tolerance than industry-standard tools and will not aim to match established services.

- **Full year competition testing:** The toolbox testing will not include an end-to-end full-year competition evaluation within the current development cycle. This would require a usable product from the beginning of the school term, allowing teams to use the service throughout their full design process.

2.2.3 Assumptions

There is an assumption of the correctness of external libraries (BFE.NET, IxMilia.Step, occt-import-js, three.js, SQL Server) used. It is assumed that they are verified by their maintainers; only our integration with them is tested.

2.3 Custom Documents

2.3.1 Fall - Interviews

The primary CD for the fall term development is interviews conducted with core stakeholders: experienced Troitsky club members and executives. These semi-structured interviews will help outline expectations and prioritization of services from knowledgeable members. These interviews will be anonymized to ensure user privacy and aggregated into usable results for the toolbox development process. The interviews will be vital in checking any assumptions made in the SRS with experienced club members.

In addition to the core interviews for the fall CD, a survey has been performed to begin the requirements elicitation process. The survey design record and raw data are stored in the project repository, and the results were used in the SRS (Azevedo et al., 2026b) to outline the priority of services and core issues members are currently facing.

2.3.2 Winter - Usability Report

The CD completed in the winter term will be a usability report, allowing for a more complete assessment of the functional usability of the toolbox. This round of testing will be performed with a more complete iteration of the product, allowing for more complete analysis of different system services' usability. Users will be gathered spanning different experience levels, both in club experience and technical experience (modelling), creating valuable data about the system's performance. Specific user functions will be tested, with

tasks that follow the G.5 usage scenarios, measured by completion rate and time on task.

This CD will further strengthen the V&V of the system, ensuring that complete functional and non-functional requirements come together into a fully usable service.

2.4 Relevant Documentation

1. **Software Requirements Specification (Azevedo et al., 2026b):** The SRS defines the functional and non-functional requirements for the Troitsky Toolbox, and is referenced in the V&V plan as the standard the system wishes to meet. The requirements set the goals for the testing areas, identifying the importance of different services, which is then reflected in the testing performed for those given systems.

2. **Hazard Analysis (Azevedo et al., 2026a):** The hazard analysis provides information regarding the primary risks and failure points of the system, pointing to the components that will require more in-depth and complex system testing. The hazard analysis also defines safety requirements for the system which are referenced in the V&V plan specifically with regard to system safety and access control correctness.

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

This sections contains the grand plan to verify and validate this project's documents and implemenation. It outlines the team, how every document, including this on, will be verified, how the implemenation will be verified, automated testing technology that will be used, how assumptions will be verified, and how contents, both internal and external will be validated.

Artificial intelligence (AI) shall be liberally employed to help us brainstorm additional ways to verify and validate all documents. However, AI frequently hallucinates and thus cannot validate or verify anything itself.

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

This project is immense and requires a lot of work to verify and validate. Peer review will be a guiding principle to catch individual errors and therefore, peer subject matter expertise shall be used instead of quality assurer and subject matter expertised being fused into one person. The following table shows the division of labour:

Name	Project Role	VnV Responsibilities
Mason Azzopardi	Project Supervisor, Back-end Developer	As project supervisor, he shall oversee all the work and guide the team to success. As a back-end developer, he shall serve as the secondary back-end expert to aid VnV related to the back-end.
Pedro Azevedo	Troitsky Liason, Front-end Developer	As Troitsky liason, he shall be the primary point of contact between this team and members of the Troitsky club. As front-end developer, he shall serve as the front-end expert to VnV related to the front-end.
Jack Easton	Back-end Developer	Serve as the primary back-end expert to aid VnV related to the back-end.
Melvin Mokhtari	Teaching Assistant	Answer administrative questions and provide feedback on course expectations and grading.
Michael Roberts	Quality Assurer	Spearhead VnV efforts. Consult with subject matter experts to draw on their expertise. Develop a robust testing suite to verify the project and consult with Troitsky Club members, especially the club president, to validate the project.
Isabella Vescio	Supervisor	An *ex officio* role stemming from her role as Troitsky Club President, she shall provide feedback so the project fits the needs and desires of the Troitsky Club. She shall involve other club members in the process as needed to assist her in this task.

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the

review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

The Software Requirements Specifications shall be verified in the following ways. In case mistakes are found or improvements are suggested, as they inevitably will, corrections will be made and the corrections shall be reviewed again. The above process is an iterative process and shall only terminate when the portion is deemed satisfactory.

- **Checklist:** Two checklists shall be developed by the Quality Assurer to verify the requirements, one for each requirement and one for the document as a whole. The former checklist shall be explicitly evaluated on each requirement to determine its quality, and will include qualities like unambiguity, verfiability, traceability, and correct spelling and grammar. The latter shall cover the pitfalls a granular analysis cannot cover, including internal consistency, completeness and documented modifiability.

- **Internal Peer Review:** Each portion of the document, especially each requirement shall be reviewed by a different group member than its author. The whole document shall be reviewed through a group-wide read-through of the document at a meeting.

- **Troitsky Review:** The Troitsky Club is our client and its members are frequently available in order for us to seek their feedback. While showing them the SRS is a fool's errand, all assumptions baked into the document shall be verified by club members through asking if the assumption is true in non-technical language they can easily understand.

- **External Peer Review:** Other capstone groups will raise issues regarding our SRS through GitHub issues. We shall address those issues, and either incorporate their feedback or dismiss it after determining their suggestion is wrong or not necessary.

- **Instructional Review:** The instructional team, primarily our assigned Teaching Assistant, will look over, comment upon, and grade the SRS. We shall seek to incorporate their feedback into our work.

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS] [Create a checklists? —SS]

The design shall stem from the requirements and shall be verified in the ways below. The verification shall be an iterative process and shall continue until the design is determined to fulfill the needs of the project.

- **Unified Modelling Language Diagrams:** All designs shall be converted from text into corresponding UML diagrams, which will facilitate verification. Many errors will be caught and fixed at the translation stage.
- **Internal Peer Review:** All designs and associated commentary shall be carefully reviewed by a different group member than the design and commentary writer. Feasability, proper decomposition, and other design principles (especially the SOLID principles) shall be guiding lights. The full list shall be contained in a checklist that the Quality Assurer will develop.
- **Assumption Verification:** All newly-encountered assumptions shall not be silently implemented into the design. Rather, they shall be verified by the Troitsky club.
- **External Review:** The group and TA assigned to peer review us shall review our design after submission and provide feedback. Furthermore, meetings with the TA will be utilized to gain more suggestions.

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

This document shall be verified in the ways outlined below. All errors and weaknesses discovered through this iterative process shall be fixed and improved, respectively. This process shall only terminate when all errors are fixed and all weaknesses are either fixed or shown to be infeasible to fix.

- **Internal Peer Review:** All sections shall be reviewed by a different team member than its author.
- **Mutation Testing:** To prevent a lazy approval, authors shall deliberately shall write at least one error in their section. Therefore, if the peer is lazy and approves a section without checking it, the author can adjure the reviewer to review the section again.
- **Assumption Verification:** Any new assumption that is absent from previous documents shall be verified by the Troitsky Club, lest the assumption is wrong.
- **Final Group Read-Through:** Before submission, this document shall be read through in a group meeting to catch holistic errors, such as contradiction, spelling and grammar errors, missing links, and incompleteness.
- **External Review:** We will recieve feedback, including issues from the external peer reviewing group in addition to grades and comments from the marking TA.

While certain verification is impossible because any verification would require its own verification, near-certainty will suffice if this document is faithfully followed.

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

The implemenation shall be throughly tested to find syntactic errors, semantic errors and weaknesses to be strengthened. Due to our meticulous development plan, requirements document, and design documents, test-driven development (TDD) is an excellent choice. The Quality Assurer shall write

these tests, drawing on his colleagues' expertise for guidance. Testing the formalized portions is low-hanging fruit, and the remaining functions will be demecrated in the design process. Of course, TDD will not catch everything so additional unit tests will be necessary after the code is written to test more cases and fill in any omissions in the pre-written tests. The requirements shall play a paramount role in determining the content of the tests. To verify the fulfillment of the requirements, each functional requirement and each objective non-functional requirement shall have at least one test verifying it. The remaining non-functional requirements will be qualitatively verified through the Quality Assurer and the Troitsky Club.

The above covers unit testing. Once the unit tests are complete and have all passed, integration and system testing shall begin. When possible, they should be written with a unit testing framework, the only difference being the test tests multiple systems and is thus an integration test. However, much of integration testing and virtually all of system testing cannot be verified automatically and must be verified manually through the Quality Assurer running the program and deeming it satisfactory. The number of parts tested for integration testing shall initially be small and incrementally rise. The last stage, will be testing the entire system (i.e. system testing).

After system testing succeeds, acceptance testing shall be done by demonstrating the software suite to the Troitsky club. Once they accept our demonstration, our suite is fit for release.

Testing is an iterative process; failures must be corrected and resubmitted. Any tests under the unit testing framework may run automatically. However, all other previous tests must be redone unless the test tests non-critical parts only and that it is painstakingly obvious that the correction did not impact the test.

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc. Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select,

you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

As stated in the Development Plan (see Expected Technology section), unit testing shall occur through an establshed C# unit testing tool, namely MS Test. Code coverage of tests shall be determined by Coverlet, an establshed testing tool. Both are open-source projects of the .NET validation.

Linting shall be done with CS Lint, freely available under the MIT license. See the Development Plan (ibid.).

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

As stated in the SRS (see S.1), our software suite shall contain one main tasks, model analysis. and many auxiliary tasks, namely a materials testing suite, a login suite, a knowledge repository (this includes both file upload and file search), and knowledge transfer tool and a broker to coordinate user tasks.

The main tasks are currently done by hand or through the graphical user interface of computer-aided design software (see SRS G.2). All calculations done by this software project shall be validated through ensuring the answers obtained from the use of this software and the current methods are the same.

Many models will be used in this process and our software and the current methods must yield the same results for each one. To prevent overfitting to our training data, some models shall be hidden during development and debugging and only utilized during testing. The number of models is to be determined, although the number of training models shall not exceed the number of test models and five of each is the current forecasted amount.

The auxiliary tasks shall be validated as follows:

- The **login suite** shall be validated against Troitsky's current list of members and their roles.
- The **knowledge transfer tool** shall be validated against Troitsky's current mechanism for creating transition documents. The club may want to alter the process for the current process is quite informal, and any alterations requests shall be meticulously followed.
- The **knowledge repository**, the **materials testing suite**, and **toolbox broker** are new capabilities which have no benchmark to validate against.

We are lucky to benefit from our primary stakeholder being almost available to us. Frequent consultation and demonstration shall be utilized to gather their feedback. Because they are our client, they will be eager to answer our questions about this project within reason. Before our first release, there shall be an extensive demonstration of our work before the Troitsky club, where we will recieve their feedback and make corrections based on it.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

The tests below are categorized by the components defined in Section S.2 of the SRS (Azevedo et al., 2026b), so that every functional requirement is covered. Each test is a system-level test of the deployed system, following

a workflow that a club member performs through the Toolbox Web App, and verifies several related requirements. The tests also verify the functional safety and security requirements that the Hazard Analysis Roadmap (Azevedo et al., 2026a) plans for the capstone. Together, they verify the model analysis correctness, data integrity, and access control identified as primary objectives in the Objectives section, while functional usability is assessed by the non-functional tests and the winter usability report.

Inputs and expected outputs refer to the symbolic constants of Table 2 and the test data of Table 3 (Symbolic Parameters appendix), shown in SMALL CAPITALS. Tests whose outputs are numerical or follow from a formula are automated, so that they can be repeated on every change; tests that depend on what a member sees on screen are manual.

4.1.1 Web Server (S.2.1)

1. T.FR.WS.1.1
   Control: Manual
   Initial State: System deployed. User not logged in.
   Input: User opens the public landing page, then attempts to open the Knowledge Repository. User then logs in with valid credentials, opens the Knowledge Repository, and logs out.
   Output: The landing page is shown without logging in. The Knowledge Repository redirects to the login page until the user logs in, and is no longer accessible after logout.
   Derivation: Verifies FR-WS-1, FR-WS-2, FR-WS-3.

4.1.2 Access Control System (S.2.2)

1. T.FR.AC.2.1
   Control: Manual
   Initial State: An executive is logged in.
   Input: The executive creates an account three times: with a password one character shorter than PASSWORD_MIN_LENGTH, with an entry from PASSWORD_BLOCKLIST, and with a password consisting solely of lowercase letters and exactly PASSWORD_MAX_LENGTH characters long. The executive then changes the account's role and team, ends its session from the server side, and deactivates it.
   Output: The first two passwords are rejected, with a message stating

that the password is too short or too common. The third is accepted, since no composition rules apply. The account always has exactly one role and at most one team. Ending the session logs the user out, and the deactivated account cannot log in.
Derivation: Verifies FR-AC-1, FR-AC-2, FR-AC-4, FR-AC-5, SR-AC-2, SR-WS-5.

2. T.FR.AC.2.2
Control: Automatic
Initial State: One account for each role of FR-AC-2, on two different teams. Each team has a current bridge model.
Input: A script requests, as each account, member management, team assignment, rule configuration, season archiving, test result review, and each team's current model.
Output: Member management, team assignment, and rule configuration are allowed only for executives and maintainers; season archiving and test result review only for executives; and a team's current model only for members of that team. Every other request is refused with an error and changes nothing.
Derivation: Verifies FR-AC-3, FR-AC-6, FR-BR-2.

4.1.3 Toolbox Broker (S.2.3)

1. T.FR.BR.3.1
Control: Manual
Initial State: System has just been started. User logged in.
Input: User opens each tool. The Material Testing Manager is then stopped, and the user requests it, then searches the Knowledge Repository.
Output: Every tool opens. The request to the stopped service returns an error message within BROKER_TIMEOUT instead of hanging, and the search returns its results.
Derivation: Verifies FR-BR-1, FR-BR-3, FR-BR-4.

4.1.4 Knowledge Repository (S.2.4)

1. T.FR.KR.4.1
Control: Manual
Initial State: User logged in. Repository contains items with known

metadata.
Input: User uploads one file of each type in COMMON_FILE_TYPES, each with a title, a category from CATEGORY_LIST, a year, a team, and tags. User then makes three uploads that each break one metadata rule: no title, a category not in CATEGORY_LIST, and no year. Finally, the user searches by keyword, filters by category, year, team, and tag, previews a file of each type in PREVIEW_FILE_TYPES, and downloads every uploaded file.
Output: Valid uploads are stored with their metadata. Each of the three invalid uploads is rejected, with a message naming the field. Searches and filters return exactly the items that match them, previews display in the browser, and each downloaded file has the same SHA-256 hash as the uploaded file.
Derivation: Verifies FR-KR-1, FR-KR-2, FR-KR-3, FR-KR-4, SR-KR-4.

2. T.FR.KR.4.2
Control: Manual
Initial State: A member has uploaded an item.
Input: The uploader edits the item, another member attempts to edit it, and an executive archives it. The repository's database server is then stopped, and a member uploads a file.
Output: Each change asks for confirmation first, and the other member's edit is refused. The archived item no longer appears in search results but can still be opened. The failed upload shows an error message, and the file is not listed once the database is restarted.
Derivation: Verifies FR-KR-5, SR-KR-2, SR-KR-5.

4.1.5 Model Analysis System (S.2.5)

1. T.FR.MA.5.1
Control: Manual
Initial State: Team member logged in. MAT_SET_A is stored.
Input: User uploads three invalid files: a PDF renamed with a `.step` extension, a STEP file with a header but no geometry, and REF_MODEL truncated to half its size. User then uploads REF_MODEL with MAT_SET_A selected, confirms the import summary, and rotates, zooms, and pans the 3D view. Finally, with MA_TIMEOUT set to MA_TIMEOUT_

TEST in the test deployment, the user uploads REF_MODEL again.
Output: Each invalid file is rejected, with a message stating that it is not a STEP file or contains no readable geometry. Before any analysis, the import summary shows the dimensions of REF_MODEL within DIMENSION_TOLERANCE and its mass within CALC_TOLERANCE of the values in Table 3. The 3D view responds to each interaction and matches the model in CAD. The last upload stops after MA_TIMEOUT_TEST with a message that the time limit was exceeded, and no model is stored.
Derivation: Verifies FR-MA-1, FR-MA-2, FR-MA-3, SR-MA-1, SR-MA-3.

2. T.FR.MA.5.2
Control: Manual
Initial State: Two teams exist, with no current models.
Input: A team member uploads REF_MODEL as a current model. A teammate, a member of the other team, and a former team member try to open it. An executive then archives the season's models, and a general member browses past designs.
Output: Only the teammate can open the current model. After archiving, it becomes a past design recording its year and team, which every member can view and filter but not modify.
Derivation: Verifies FR-MA-4, FR-MA-5, FR-MA-6, FR-AC-6.

4.1.6 Performance Analysis System (S.2.5.1)

1. T.FR.PA.6.1
Control: Automatic (results) and Manual (displayed assumptions)
Initial State: REF_MODEL, MAT_SET_A, and MAT_SET_B are stored.
Input: A performance analysis of REF_MODEL is run with MAT_SET_A, then with MAT_SET_B, then with the calculation library made to return a non-finite stress.
Output: For each material set, the failure load and load-to-mass ratio are within CALC_TOLERANCE of the expected values in Table 3, and the result names the material set used. The assumptions and limitations are shown with the results. For the non-finite stress, an error is shown instead of an estimate.
Derivation: Verifies FR-PA-1, FR-PA-2, FR-PA-3, FR-PA-4, FR-

MT-4, SR-PA-1. Expected values follow from SRS S.7.2, as derived in Table 3.

4.1.7 Rule Compliance System (S.2.5.2)

1. T.FR.RC.7.1
   Control: Manual (rule set editing) and Automatic (compliance checks)
   Initial State: Executive logged in. The compliance models of Table 4 are stored.
   Input: The executive imports RULESET_TEST, adds a rule, edits its limit, and removes it. A script then runs a compliance check of each compliance model against RULESET_TEST.
   Output: The rule set is stored under its year, and after each edit lists exactly the expected rules. RC_PASS and RC_LIMIT pass every rule, and each other compliance model fails only the rule it is named for, showing its measured value and the allowed range. The materials rule is listed as a manual check for every model.
   Derivation: Verifies FR-RC-1, FR-RC-2, FR-RC-3, FR-RC-4, FR-RC-5. Expected results follow from the `RuleComplianceCheck` predicate of SRS S.7.3, whose limits are inclusive.

4.1.8 Material Testing Manager (S.2.6)

1. T.FR.MT.8.1
   Control: Manual
   Initial State: MAT_SET_B is stored as the previous year's results.
   Input: User enters the first specimen of MAT_SET_A through the form, first without a grain orientation, then with it. User imports the remaining specimens of MAT_SET_A from MT_IMPORT_FILE, which also contains four invalid rows. User views the comparison with MAT_SET_B, and an executive flags the 840 N specimen of MAT_SET_A as invalid.
   Output: The entry without a grain orientation is rejected, and the second is stored. The four invalid rows are rejected, each identified by its row and field, and the valid rows are stored. The comparison shows the mean and sample standard deviation of each set given in Table 3. After flagging, the strength of MAT_SET_A is recomputed to the flagged value in Table 3, and the flagged specimen remains stored, marked invalid.

Derivation: Verifies FR-MT-1, FR-MT-2, FR-MT-3, FR-MT-4, FR-MT-5, SR-MT-2.

4.1.9 Knowledge Transfer Manager (S.2.7)

1. T.FR.KT.9.1
   Control: Manual
   Initial State: Executive logged in. A past design is stored.
   Input: The executive opens their exit document template, saves a draft, and submits it three times: with a reference to a model identifier that does not exist, with a reference to the stored past design while the repository's database server is stopped, and with the same reference once it is restarted. Another executive then views the outstanding exit documents.
   Output: A template exists for each executive position and for team leads, and the draft keeps its content. The invalid reference is rejected. While the database is stopped, the document is not marked as published and the executive is notified. Once saved, it is published and can be found by search, and the outstanding list no longer includes the executive.
   Derivation: Verifies FR-KT-1, FR-KT-2, FR-KT-4, FR-KT-5, SR-KT-2, SR-KT-5.

2. T.FR.KT.9.2
   Control: Manual
   Initial State: An executive holds a position with a role record.
   Input: The executive and another member each try to update the role record. The executive's account is then deactivated, first without a successor, then after a successor is assigned.
   Output: Only the executive's update is saved. Deactivation is blocked until a successor is assigned, after which the successor can update the role record.
   Derivation: Verifies FR-KT-3, FR-KT-4, SR-KT-3.

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention re-

porting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Webserver

Communication Encryption

1. T.NFR.WS.2.1
   Type: Automatic

   Initial State: Troitsky Toolbox webserver is active and able to receive packets.

   Input/Condition: A script inputs logs into the webserver and makes 1000 requests to the webserver, half with secure protocols half with insecure.

   Output/Result:
   `Pass:` All modern and secure protocols and encryptions are negotiated and all outdated and insecure protocols are rejected.
   `Fail:` Any modern and secure protocols and encryptions are rejected or outdated and insecure protocols are negotiated.

   How the test will be performed: Tools like Nmap or, Qualys SSL Labs, will scan the packet and observe the encryption protocol and cyphers. Only negotiate with packets that have secure and modern protocols (TLS 1.2 & 1.3), reject older protocols (SSLv3, TLS 1.0, & TLS 1.1) and weak cyphers (RC4, DES, or NULL)

2. T.NFR.WS.2.2
   Type: Automatic.

   Initial State: Webserver is running and currently stable

   Input/Condition: Updated system code

   Output/Result: CI/CD test pass or fail

   How the test will be performed: Scripts written to assert

   (a) Requests to `http://...` results in a redirect to `https://...`
   (b) Requests to `https://...` succeeds and returns a 200 OK
   (c) A request using a deliberately downgrade TLS version throws a connection reset/SSL handshake error.

3. T.NFR.WS.2.3
   Type: Dynamic

   Initial State: Webserver is running and currently stable

   Input/Condition: Webserver Address.

   Output/Result: Potential vulnerabilities like broken or missing encryptions.

   How test will be preformed: Using a tool like the Open Worldwide Application Security Project (OWASP) Zed Attack Proxy (ZAP) will actively probe a running web server to find vulnerabilities, including broken or missing encryption.

Webserver Timeout

1. T.NFR.WS.3.1
   Type: Automatic

   Initial State: There are currently uses with sessions open in the web server

   Input/Condition: Test users has been detected as inactive.

   Output/Result:
   `Pass:` The test user is inactive for 30 minutes (SESSION_TIMEOUT) and they are timed out.

`Fail:` The test user is inactive for 30 minutes and isn't timed out or they are inactive for less than 30 minutes and are timed out.

How the test will be performed:

(a) A script will have 100 test users log into the toolbox and be inactive

(b) The script will read the WebServer logs and start a secondary timer for each user when the Webserver starts the inactivity timer for each user.

(c) Half the test users will remain inactive for over 30 minutes and the other half will randomly become active before the 30 minute window (with a few users designed to become active at edge cases like the start of the inactivity timer and the end.).

(d) The script pauses each users secondary timer when they are timed out or become active.

(e) The test passes if all secondary timers align with the 30 minute inactivity timeout window.

4.2.2 Access Control System

Security Credential and Role Alteration Ensurance

1. T.NFR.AC.1.1
   Type: Automatic

   Initial State: Webserver is running and currently stable

   Input/Condition: Updates to the Access Control System is pushed to the project.

   Output/Result:
   `Pass:` Sample passwords are stored in the correct positions within the hash table
   `Fail:` Sample password are stored in the incorrect positions within the hash table or don't exist within the hash table.

   How test will be preformed:

   (a) Create a new test user with a known password (e.g., TestPassword123!). Query the database table where users are stored.

(b) Create two users with the same password, query the database for both users.

2. T.NFR.AC.1.2
   Type: Automatic

   Initial State: A user is actively logged into the system with a specific role and is currently interacting with data they are authorized to see.

   Input/Condition: While the user is still active, an administrator alters the user's role, which removes their permission to view the data they were just interacting with.

   Output/Result:
   `Pass:` The system denies access on the user's very next action without requiring them to log out and log back in.
   `Fail:` The user is still able to view/edit the restricted data, or the system waits for a session timeout/re-login before enforcing the new role.

   How test will be preformed:

   (a) A script logs into the system as a test user and confirms they can access a restricted feature

   (b) A separate admin process changes the test user's role in the system

   (c) Without logging out the test user immediately attempts to use the restricted feature again.

   (d) Script observes the systems response to ensure access is instantly blocked.

4.2.3 Toolbox Broker

Extensibility and Overhead Management

1. T.NFR.TB.1.1
   Type: Automatic / Manual

   Initial State: The Toolbox Broker is running normally and successfully routing requests to existing, standard services. A new, isolated "mock" service has been created but is not yet known to the system.

Input/Condition: The new mock service is registered with the Toolbox Broker

Output/Result:
`Pass:` Broker successfully routes the request to the new service and returns the response, with no older services function being compromised.
`Fail:` Broker fails to find new service or existing services are broken by the addition of the mock service.

How test will be preformed:

(a) A script or test deploys a mock service

(b) The mock service is registered with the Broker at run time.

(c) A test request is sent to the new service to confirm it connects.

(d) A test request is sent to existing services to ensure backward compatibility and system stability remain intact.

2. T.NFR.TB.2.1
Type: Automatic

Initial State: The Toolbox Broker and a target services are running under normal system conditions.

Input/Condition: A script sends a batch of requests directly to the target service to establish a baseline response time. Then it sends an identical batch of requests to the same service but routes them through the Toolbox Broker.

Output/Results:
`Pass:` The difference between the baseline response time and the broker response time is strictly MAX_BROKER_OVERHEAD (100 ms) or less for all test requests.
`Fail:` The presence of the broker delays the requests by more than 100ms

How the test will be preformed:

(a) An automate script sends 1000 requests directly to a test service and records the average response time

(b) The script sends 1000 identical requests through the toolbox broker to the same test service and records the average response time.

(c) The scrip then calculates $direct_request_time - broker_request_time$ for every request.

(d) The test pass if every calculation is below 100ms.

Request Logging

1. T.NFR.TB.3.1
   Type: Automatic

   Initial State:The Toolbox Broker is running, and its logging mechanism is accessible and at a known starting point.

   Input/Condition: A specific, authenticated user intentionally triggers two distinct actions through the Broker: one request that is known to succeed (e.g., fetching a public list) and one request that is known to fail (e.g., attempting an invalid or unauthorized operation)

   Output/Result:
   `Pass:` The Broker's log output displays two new distinct entries. Each entry explicitly contains: the user identifier, the operation requested, a timestamp, and a clear success/fail boolean or status code.
   `Fail:` No logs are generated, or the logs are missing one of the four required pieces of information (e.g., it logs the error but forgets to log who triggered it).

   How the test will be preformed:

   (a) A script authenticates the test user

   (b) The script sends a valid and invalid request

   (c) The script reads the brokers logs

   (d) The script parses the most recent log entries and asserts that all four required fields (Member, Operation, Time, Success/Failure Status) are present and correctly recorded for both the successful and failed attempts.

4.2.4 Knowledge Repository

Search Responsiveness

1. T.NFR.KR.1.1
   Type: Automatic (Performance)

   Initial State: The Knowledge Repository database is populated with 10,000 mock items (documents, bridge models, test results).

   Input/Condition: An automated script sends varied search queries (by keyword, year, team, tag) to the repository.

   Output/Result:
   `Pass:` All search results are returned and displayed to the client in under 2.0 seconds (MAX_SEARCH_TIME).
   `Fail:` Any search query takes longer than 2.0 seconds.

   How the test will be performed: A load-testing script will fire a suite of search queries against the API. The script will measure the round-trip time from request to response payload delivery and assert it falls below the threshold.

Data Durability and External Backup

1. T.NFR.KR.2.1
   Type: Automatic / Manual (Infrastructure)

   Initial State: The Knowledge Repository contains newly uploaded files and database records. The backup schedule is active.

   Input/Condition: The BACKUP_INTERVAL (7 days) passes, triggering an automated backup. A catastrophic server failure is then simulated by wiping the active database.

   Output/Result:
   `Pass:` A full backup exists on an isolated, external platform, and the system can be fully restored with no more than 7 days of data lost.
   `Fail:` The backup fails to trigger, is stored on the same vulnerable server, or fails restoration.

   How the test will be performed: The DevOps/infrastructure pipeline will verify the backup cron job executes. A developer will manually trigger a restoration in a staging environment using the external backup file to ensure data integrity is intact.

4.2.5 Model Analysis System

Strict STEP File Upload

1. T.NFR.MA.1.1
   Type: Automatic

   Initial State: The user is on the model upload page.

   Input/Condition: The user attempts to upload a valid STEP file, an invalid .stl file, and a malicious executable disguised as STEP.exe.

   Output/Result:
   `Pass:` The `.stp` file is accepted. The other formats are strictly rejected by the server with a clear error message.
   `Fail:` The system accepts non-STEP files or crashes attempting to parse them.

   How the test will be performed: An API testing script will Post various file types and extensions to the upload endpoint and assert the HTTP response codes (200 OK for STEP, 400/415 for others).

3D Visualization Load Time

1. T.NFR.MA.2.1
   Type: Automatic (Performance)

   Initial State: A valid, typical club bridge STEP file is stored in the repository.

   Input/Condition: The user clicks to view the 3D visualization of the model.

   Output/Result:
   `Pass:` The interactive 3D model fully renders in the browser in under 30 seconds (MAX_MODEL_LOAD_TIME).
   `Fail:` The model takes longer than 30 seconds to render or hangs indefinitely.

   How the test will be performed: A browser automation tool (like Selenium or Playwright) will click the model link, start a timer, and stop the timer when the 3D canvas object registers as fully loaded and interactive.

Extraction Accuracy

1. T.NFR.MA.3.1
   Type: Automatic

   Initial State: A standardized test STEP file with exact, mathematically known dimensions is uploaded.

   Input/Condition: The Model Analysis System parses the file and extracts its dimensions.

   Output/Result:
   `Pass:` The extracted dimensions match the source model to within a 0.5 mm tolerance (DIMENSION_TOLERANCE).
   `Fail:` The dimensions deviate by more than 0.5 mm.

   How the test will be performed: A unit/integration test will pass the specific STEP file to the parsing library, retrieve the output data struct, and assert the length/width/height values fall within actual value $\pm$ 0.5mm.

4.2.6 Performance Analysis System

Estimate Accuracy

1. T.NFR.PA.1.1
   Type: Automatic (Algorithm Validation)

   Initial State: The system holds STEP models of past club bridges, alongside the actual recorded failure loads from past competitions.

   Input/Condition: The Performance Analysis System runs a failure load estimate on the historical models.

   Output/Result:
   `Pass:` The estimated failure load is within 25% (FAILURE_LOAD_TOLERANCE) of the real-world recorded value.
   `Fail:` The estimate diverges by more than 25%.

   How the test will be performed: A validation script will run the FEA algorithm against a suite of 5-10 historical bridge models. It will compare the output to the known real-world data and calculate the percentage error.

Computation Time

1. T.NFR.PA.2.1
   Type: Automatic (Performance)

   Initial State: A typical club bridge model is loaded.

   Input/Condition: The user triggers the performance analysis.

   Output/Result:
   `Pass:` The analysis completes and returns the failure load and metrics in under 30 seconds (MAX_ANALYSIS_TIME).
   `Fail:` The computation exceeds 30 seconds.

   How the test will be performed: An automated script triggers the analysis API and measures the response time.

Extensibility of Metrics

1. T.NFR.PA.3.1
   Type: Manual (Architecture check)

   Initial State: The PA system codebase is open in a development environment.

   Input/Condition: A developer registers a new dummy metric calculation (e.g., "Bridge Flexibility Score") in the codebase.

   Output/Result:
   `Pass:` The new metric appears in the output payload without requiring alterations to the core code calculating Failure Load or Load-to-Mass ratio.
   `Fail:` Modifying/adding a metric breaks existing metric outputs.

   How the test will be performed: Verified via Code Review and unit tests during the addition of a new metric to ensure the Open/Closed Principle of software architecture is maintained.

4.2.7 Rule Compliance System

Check Correctness (No False Passes)

1. T.NFR.RC.1.1
   Type: Automatic

Initial State: A test suite of bridge models exists. Some are perfectly compliant, others have specific, known rule violations (e.g., too heavy, clearance too low).

Input/Condition: The Rule Compliance System evaluates the entire suite against the active rule set.

Output/Result:
`Pass:` Every single non-compliant bridge is correctly flagged as a Fail.
`Fail:` The system gives a false pass to a bridge that violates a rule.

How the test will be performed: The CI/CD pipeline runs a script passing the test suite through the compliance engine. The test asserts that the output array of the non-compliant models strictly contains `isValid: false`.

Rule Updatability (No Code Changes)

1. T.NFR.RC.2.1
   Type: Manual

   Initial State: The system is live. An executive user is logged in.

   Input/Condition: The competition releases a new rule. The executive updates this value via the application UI.

   Output/Result:
   `Pass:` The new rule is applied instantly to the next model evaluated. No source code modification, recompilation, or server restart was required.
   `Fail:` A developer is required to change the rule in the codebase.

   How the test will be performed: A tester logs in as an Executive, navigates to the Rules configuration page, alters a parameter, saves, and immediately runs a compliance check to verify the new parameter is actively enforced.

4.2.8 Material Testing Manager

Result Import and Export

1. T.NFR.MT.1.1
   Type: Automatic

Initial State: The user is on the Material Testing interface.

Input/Condition: The user uploads a standardized CSV containing 100 rows of test data. Later, the user clicks "Export to CSV".

Output/Result:
`Pass:` The file imports successfully with all 100 rows accurately reflected in the database. The exported CSV matches the data exactly.
`Fail:` The import fails, truncates data, or locks the data inside the system preventing export.

How the test will be performed: A script uploads a mock CSV, queries the DB to verify row count and data integrity, then hits the export API endpoint and compares the downloaded file's checksum/content against the original.

Traceability

1. T.NFR.MT.2.1
   Type: Automatic

   Initial State: Material properties have been aggregated from a batch of test results.

   Input/Condition: A user queries the aggregated material property (e.g., Average Glue Strength).

   Output/Result:
   `Pass:` The returned data payload includes direct database references/links to the exact individual test result IDs used to calculate that average.
   `Fail:` The aggregate is presented as a static number with no link to its raw source data.

   How the test will be performed: An API test requests an aggregated property and asserts that the response JSON contains a source_test_ids array connecting it to the raw entries.

4.2.9 Knowledge Transfer Manager

Findability

1. T.NFR.KT.1.1
   Type: Manual (Usability)

Initial State: A brand new user (simulating a new executive) logs into the system for the first time.

Input/Condition: The user is tasked with finding the role record for the "VP of Design".

Output/Result:
`Pass:` The user successfully navigates the UI and opens the document in under 2 minutes (MAX_ROLE_SEARCH_TIME).
`Fail:` The user takes longer than 2 minutes or gives up.

How the test will be performed: Conducted during the Winter CD Usability Testing phase. A tester observes a user with a stopwatch to measure the time to successful task completion.

Longevity (PDF Export)

1. T.NFR.KT.2.1
   Type: Automatic

   Initial State: A completed exit document exists in the system.

   Input/Condition: The user clicks "Export to PDF".

   Output/Result:
   `Pass:` The server generates and downloads a `.pdf` file containing the document's text and metadata cleanly formatted.
   `Fail:` The export fails or produces an unreadable file.

   How the test will be performed: An automated script calls the PDF generation endpoint, checks the response header for application/pdf, and validates the file size is > 0 bytes.

4.2.10 System-Wide Requirements

Browser Support

1. T.NFR.SYS.1.1
   Type: Manual / Automated (Cross-Browser)

   Initial State: The web application is deployed to a staging environment.

   Input/Condition: A script (or tester) executes the core workflows (Login, Upload Model, Run Analysis) on the latest versions of Chrome, Firefox, Edge, and Safari.

Output/Result:
`Pass:` The application functions perfectly on all 4 browsers with no broken layouts or JS errors.
`Fail:` A core feature fails to execute on one or more of the specified browsers.

How the test will be performed: End to end testing framework configured to run the test suite across multiple browser engines.

Capacity and Concurrency

1. T.NFR.SYS.2.1
   Type: Automatic (Stress Test)

   Initial State: The application is running under normal conditions.

   Input/Condition: 50 simulated users (MIN_CONCURRENT_USERS) log in and perform varied tasks (searching, uploading, routing requests) simultaneously.

   Output/Result:
   `Pass:` All requests succeed, response times stay within their defined limits, and no thread-safety/concurrency errors occur.
   `Fail:` The server crashes, drops requests, or exhibits race conditions.

   How the test will be performed: A load testing tool spins up 50 concurrent virtual users. The script asserts zero 500-level HTTP errors and verifies that user data does not bleed between active sessions.

Service Independence

1. T.NFR.SYS.3.1
   Type: Automatic (Chaos Testing)

   Initial State: All services are running normally.

   Input/Condition: The Model Analysis System container is deliberately crashed/taken offline. A user then attempts to search the Knowledge Repository.

   Output/Result:
   `Pass:` The Knowledge Repository returns search results normally. The broker gracefully handles the MA failure by returning an isolated error

for MA tasks only.
`Fail:` The whole application goes down or the broker hangs indefinitely.

How the test will be performed: A CI/CD script kills the MA service process, then sends HTTP GET requests to the KR service and asserts a `200 OK` response.

Maintainability

1. T.NFR.SYS.4.1
   Type: Manual (Developer Onboarding)

   Initial State: A new Software Subteam recruit has just been granted repository access.

   Input/Condition: The recruit follows the documentation (e.g., `README.md`, Docker setup guides) to build the app locally, write a minor UI text change, and deploy it.

   Output/Result:
   `Pass:` The new member successfully completes the full cycle in under 1 week (MAX_ONBOARDING_TIME) with minimal external help.
   `Fail:` Documentation is missing, outdated, or the environment is too complex to set up within a week.

   How the test will be performed: A real-world onboarding trial where a new CAS student is timed and monitored while setting up the project using only the written documentation.

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

Table 1: Traceability between System Tests, SRS Requirements, and Hazard Analysis Requirements

Test Case ID	**SRS Requirement**	**Hazard Analysis Requirement**
T.FR.WS.1.1	FR-WS-1, FR-WS-2, FR-WS-3	N/A
T.FR.AC.2.1	FR-AC-1, FR-AC-2, FR-AC-4, FR-AC-5	SR-AC-2, SR-WS-5
T.FR.AC.2.2	FR-AC-3, FR-AC-6, FR-BR-2	N/A
T.FR.BR.3.1	FR-BR-1, FR-BR-3, FR-BR-4	N/A
T.FR.KR.4.1	FR-KR-1, FR-KR-2, FR-KR-3, FR-KR-4	SR-KR-4
T.FR.KR.4.2	FR-KR-5	SR-KR-2, SR-KR-5
T.FR.MA.5.1	FR-MA-1, FR-MA-2, FR-MA-3	SR-MA-1, SR-MA-3
T.FR.MA.5.2	FR-MA-4, FR-MA-5, FR-MA-6, FR-AC-6	N/A
T.FR.PA.6.1	FR-PA-1, FR-PA-2, FR-PA-3, FR-PA-4, FR-MT-4	SR-PA-1
T.FR.RC.7.1	FR-RC-1, FR-RC-2, FR-RC-3, FR-RC-4, FR-RC-5	N/A
T.FR.MT.8.1	FR-MT-1, FR-MT-2, FR-MT-3, FR-MT-4, FR-MT-5	SR-MT-2
T.FR.KT.9.1	FR-KT-1, FR-KT-2, FR-KT-4, FR-KT-5	SR-KT-2, SR-KT-5
T.FR.KT.9.2	FR-KT-3, FR-KT-4	SR-KT-3
T.NFR.WS.2.1	NFR-WS-2	N/A
T.NFR.WS.2.2	NFR-WS-2	N/A
T.NFR.WS.2.3	NFR-WS-2	N/A

Test Case ID	SRS Requirement	Hazard Analysis Requirement
T.NFR.WS.3.1	NFR-WS-3	N/A
T.NFR.AC.1.1	NFR-AC-1, NFR-AC-2	N/A
T.NFR.AC.1.2	NFR-AC-1, NFR-AC-2	SR-AC-1
T.NFR.TB.1.1	NFR-TB-1	N/A
T.NFR.TB.2.1	NFR-TB-2	N/A
T.NFR.TB.3.1	NFR-TB-3	N/A
T.NFR.KR.1.1	NFR-KR-1	N/A
T.NFR.KR.2.1	NFR-KR-2	SR-KR-1 (External Backup)
T.NFR.MA.1.1	NFR-MA-1	N/A
T.NFR.MA.2.1	NFR-MA-2	N/A
T.NFR.MA.3.1	NFR-MA-3	N/A
T.NFR.PA.1.1	NFR-PA-1	N/A
T.NFR.PA.2.1	NFR-PA-2	N/A
T.NFR.PA.3.1	NFR-PA-3	N/A
T.NFR.RC.1.1	NFR-RC-1	N/A
T.NFR.RC.2.1	NFR-RC-2	N/A
T.NFR.MT.1.1	NFR-MT-1, NFR-MT-3	N/A
T.NFR.MT.2.1	NFR-MT-2	N/A
T.NFR.KT.1.1	NFR-KT-1	N/A
T.NFR.KT.2.1	NFR-KT-2	N/A
T.NFR.SYS.1.1	NFR-SYS-1	N/A
T.NFR.SYS.2.1	NFR-SYS-2	SR-WS-3 (Thread Safety)
T.NFR.SYS.3.1	NFR-SYS-3	N/A
T.NFR.SYS.4.1	NFR-SYS-4	SR-WS-1 (Code Testing)

FR-KT-5 is a Could requirement in SRS S.5, implemented only if time allows, so its part of T.FR.KT.9.1 is performed only if it is implemented.

SR-SYS-1 to SR-SYS-3 constrain how the system is built and deployed rather than its behaviour, so they are verified as part of implementation

verification rather than by system tests. The safety and security requirements that the Hazard Analysis Roadmap delays until after the capstone are not tested: SR-KR-3, SR-MA-2, SR-RC-1, SR-KT-1, SR-KT-4, SR-MT-1, and SR-MT-3.

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

Unit tests are an integral part of the verification of the implementation of this project. Below is a sketch of the unit tests that will be employed in this software project.

5.1 Unit Testing Scope

The crown jewel of this project is the model analysis, because all other services offered are pretty simple. The model analysis service contains five (5) modules: (a) an upload module to upload the STEP file, (b) an external computer-aided design (CAD) module to perform technical engineering calculations, (c) the bridge failure module, (d) the rule compliance module, and (e) the scoring module. The upload module is very straightforward; it can be done with standard file integrity testing. The validation of the calculations of the external CAD module is beyond the software scope of this project; rather, they shall be shown to be accurate by comparing results against the current standards process Troitsky uses and deeming it satisfactory if they are the same for several STEP files, including files purposely hidden from the training data. The last three modules shall be verified through unit testing,

having been formalized in the SRS (see S.7).

The other services are the log-in, the materials testing database, the knowledge repository (both adding new files and searching existing files), and the knowledge transfer service to facilitate the creation of transition documents. All these tasks will contain are a login suite, page rendering, role-based access control, CRUD (create, read, update, delete) operations for a database or a shared file store, a form to submit information, and a naive search engine (a simple LIKE-based search is almost certainly sufficient for now). These problems are not novel and be quickly implemented and verified.

Therefore, while unit testing should exist for them, a few will suffice for each. Therefore, the vast majority of unit testing should be directed at the last three sub-modules of the model analysis service.

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

Below is a sketch of automated tests for the functional requirements. First are the three major modules and below them are the auxiliary services.

For the first two tests, two phyla of tests shall be conducted, depressed tests with dummy data, and actual tests with a real STEP file. The former are white box, as the input is artificial and the latter are black box, as the output is based on a bona fide STEP file. Inputting artificial data is the domain of the developers for testing. A real user should input a raw STEP file and see the output of the system's analysis of it, because calculating and inputting intermediate results an external CAD library would do is highly error-prone.

Due to the straightforwardness of the auxiliary services above, each service shall contain no more than two (2) examples, of unit tests. There may be more in the actual unit testing suite, but this document serves as a sketch, not a complete guide, for the suite.

5.2.1 Bridge Failure Module

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

This module determines whether a bridge fails, i.e., whether there is an element in the bridge whose factor of safety is less than one. The examples below are illustrative of the test types, in reality there will be several for each type.

1. Artificial, Bridge Succeeds

   Type: Automatic

   Initial State: System requests artificial data for an STEP model to analyze.

   Input: This bridge contains two elements. Element e_a is of a material m_a where $\mu_{strength}(m_a) = 5$ and $\sigma_{API}(e_a) = 4$ and Element e_b is of a material m_b where $\mu_{strength}(m_b) = 3$ and $\sigma_{API}(e_b) = 2$.

   Output: $BridgeFailure \equiv false$

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$
\begin{aligned}
BridgeFailure &\equiv (\exists e : \frac{\mu_{\text{strength}}(\text{elementMaterial}(e))}{\sigma_{\text{API}}(e)} < 1) \\
&\equiv \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_a))}{\sigma_{\text{API}}(e_a)} < 1 \ \vee \ \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_b))}{\sigma_{\text{API}}(e_b)} < 1 \\
&\equiv \frac{5}{4} < 1 \ \vee \ \frac{3}{2} < 1 \\
&\equiv \text{false} \vee \text{false} \\
&\equiv \text{false}
\end{aligned}
$$

   How test will be performed: Through MsTest within the .NET solution used for implementation

2. Artificial, Bridge Fails

   Type: Automatic

   Initial State: System requests artificial data for an STEP model to analyze.

   Input: This bridge contains two elements. Element e_a is of a material m_a where $\mu_{strength}(m_a) = 5$ and $\sigma_{API}(e_a) = 4$ and Element e_b is of a material m_b where $\mu_{strength}(m_b) = 3$ and $\sigma_{API}(e_b) = 4$.

   Output: $BridgeFailure \equiv true$

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$
\begin{aligned}
BridgeFailure &\equiv (\exists e : \frac{\mu_{\text{strength}}(\text{elementMaterial}(e))}{\sigma_{\text{API}}(e)} < 1) \\
&\equiv \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_a))}{\sigma_{\text{API}}(e_a)} < 1 \ \vee \ \frac{\mu_{\text{strength}}(\text{elementMaterial}(e_b))}{\sigma_{\text{API}}(e_b)} < 1 \\
&\equiv \frac{5}{4} < 1 \ \vee \ \frac{3}{4} < 1 \\
&\equiv \text{false} \vee \text{true} \\
&\equiv \text{true}
\end{aligned}
$$

   How test will be performed: Through MsTest within the .NET solution used for implementation

3. Natural

   Type: Automatic

   Initial State: System requests an STEP file that models a bridge

   Input: The STEP file

   Output: If there is an element e where

$$\frac{\mu_{\text{strength}}(\text{elementMaterial}(e))}{\sigma_{\text{API}}(e)} < 1$$

   $BridgeFailure \equiv true$. Otherwise, $BridgeFailure \equiv false$.

Test Case Derivation: From the formalization in (S.7.2 (1)). The external CAD library will return $m_e = elementMaterial(e)$ and $\sigma_{API}(e)$ for each element present in the bridge model. The materials testing module will be invoked to find $\mu_{\text{strength}(m_e)}$. Then, this module will have gained its input to perform its calculations and will yield outputs consistent with the formula provided. For more information, see the above two artificial test cases.

How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.2 Rules Compliance Module

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

This module determines whether a bridge complies with the rules for the Troitsky competition. Because the competition rules change each year, the fields denoting rule-based constraints are variable so they can be easily updated. The examples below are illustrative of the test types, in reality there will be several for each type.

For brevity, the example rules are the same in all three test cases. The maximum weight is 7 kg, the minimum length is 175 m, the maximum length is 225 m, the minimum width is 45 m, the maximum width is 60 m and the minimum clearance height is 45 m. Furthermore, let an iB (in bounds) function be defined as

$$iB(val, lower, upper) \equiv lower \leq val \land val \leq upper$$

Lastly, variables shall be in the form of $qualQty$ where $qual \in$ (val, min, max) and $Qty \in$ (Wt, Ln, Wd, Ht). The variable shall be understood as "the $qual$ for the Qty". The items in the $qual$ (qualifiers) set are short for value, minimum and maximum respectively, while the items in the Qty (quantities) set are short for weight, length, width and height respectively. If no qualifier is there (e.g. Ln), an implicit qualifier of val shall be assumed.

1. Artificial, Rules Compliant

Type: Automatic

Initial State: System requests artificial data for an STEP model to analyze.

Input: This bridge weighs 5 kg, has a length of 200 m, a width of 50 m and a clearance height of 50 m.

Output: $isValid \equiv true$

Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$\begin{aligned} isValid &\equiv iB(Wt, 0, maxWt) \wedge iB(Ln, minLn, maxLn) \wedge \\ &\quad iB(Wd, minWd, maxWd) \wedge Ht < maxHt \\ &\equiv iB(5, 0, 7) \wedge iB(175, 200, 225) \wedge iB(45, 50, 60) \wedge 45 \leq 50 \\ &\equiv true \wedge true \wedge true \wedge true \\ &\equiv true \end{aligned}$$

How test will be performed: Through MsTest within the .NET solution used for implementation

2. Artificial, Rules Non-Compliant

Type: Automatic

Initial State: System requests artificial data for an STEP model to analyze.

Input: This bridge weighs 5 kg, has a length of 200 m, a width of 61 m and a clearance height of 50 m.

Output: $isValid \equiv false$

Test Case Derivation: [Justify the expected value given in the Output field —SS]

$$
\begin{aligned}
isValid &\equiv iB(Wt, 0, maxWt) \wedge iB(Ln, minLn, maxLn) \wedge \\
&\quad iB(Wd, minWd, maxWd) \wedge Ht < maxHt \\
&\equiv iB(5, 0, 7) \wedge iB(175, 200, 225) \wedge iB(45, 61, 60) \wedge 45 \leq 50 \\
&\equiv true \wedge true \wedge false \wedge true \\
&\equiv false
\end{aligned}
$$

How test will be performed: Through MsTest within the .NET solution used for implementation

3. Natural

   Type: Automatic

   Initial State: System requests an STEP file that models a bridge

   Input: The STEP file

   Output: If,

   $iB(Wt, 0, maxWt) \wedge iB(Ln, minLn, maxLn) \wedge iB(Wd, minWd, maxWd) \wedge Ht < maxHt$

   $isValid \equiv true$. Otherwise, $isValid \equiv false$.

   Test Case Derivation: From the formalization in (S.7.2 (2)). The external CAD library will return the bridge's weight, length width and height. Then, this module will have gained its input to perform its calculations and will yield outputs consistent with the formula provided. For more information, see the above two artificial test cases.

   How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.3 Bridge Scoring Module

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

This module determines the score of a bridge. The score is based on the formula in SRS section S.7.3 (3).

For brevity, there will be two hard constraints, two objectives and two soft constraints. As the real calculations required to determine these values are yet to be decided, a natural unit test would be inappropriate. The score shall not be less than zero. In all three test cases, $w_1 = 5$, $w_2 = 3$, $p_1 = 400$, and $p_2 = 200$. See S.7.3 for the meanings of the abbreviations used here.

1. Violates Hard Constraint

   Type: Automatic

   Initial State: System requests artificial data to run scoring module.

   Input: $r_1 = 0$, $r_2 = 1$, $s_1 = 100$, $s_2 = 50$, $b_1 = 0$, $b_2 = 1$.

   Output: $W = 0$

   Test Case Derivation:

$$\begin{aligned} W &= max(r_1 r_2(w_1 s_1 + w_2 s_2 - (b_1 p_1 + b_2 p_2)), 0) \\ &= max(0(1)(5 * 100 + 3 * 50 - (0 * 400 + 1 * 200)), 0) \\ &= max(0(450), 0) \\ &= 0 \end{aligned}$$

   How test will be performed: Through MsTest within the .NET solution used for implementation

2. Normal

   Type: Automatic

   Initial State: System requests artificial data to run scoring module.

   Input: $r_1 = 1$, $r_2 = 1$, $s_1 = 100$, $s_2 = 50$, $b_1 = 0$, $b_2 = 1$.

   Output: $W = 450$

   Test Case Derivation:

$$
\begin{aligned}
W &= max(r_1 r_2 (w_1 s_1 + w_2 s_2 - (b_1 p_1 + b_2 p_2)), 0) \\
&= max(1(1)(5 * 100 + 3 * 50 - (0 * 400 + 1 * 200)), 0) \\
&= max(1(450), 0) \\
&= 450
\end{aligned}
$$

How test will be performed: Through MsTest within the .NET solution used for implementation

3. Below Floor of Zero

Type: Automatic

Initial State: System requests artificial data to run scoring module.

Input: $r_1 = 0$, $r_2 = 1$, $s_1 = 50$, $s_2 = 100$, $b_1 = 1$, $b_2 = 1$.

Output: $W = 0$

Test Case Derivation:

$$
\begin{aligned}
W &= max(r_1 r_2 (w_1 s_1 + w_2 s_2 - (b_1 p_1 + b_2 p_2)), 0) \\
&= max(1(1)(5 * 50 + 3 * 100 - (1 * 400 + 1 * 200)), 0) \\
&= max(1(-50), 0) \\
&= 0
\end{aligned}
$$

How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.4 Log-in

This module handles an attempt by any person to demonstrate they are the person associated with a given user account. Because single sign-on (SSO) shall be employed, the system shall recieve a confirmation a user is the owner of a specific email address, almost always a McMaster one, not a username and password.

1. Successful Attempt

   Type: Automatic

   Initial State: Session not logged in

   Input: SSO confirmation of an email that matches an active account in system

   Output: Session logged in as that account

   Test Case Derivation:

   - System performs parametrized query on the Users database looking for user accounts that match the inputted confirmation's email
   - System finds one active account in the system that fits the query
   - System logs in session with that active account

   How test will be performed: Through MsTest within the .NET solution used for implementation

2. Failed Attempt

   Type: Automatic

   Initial State: Session not logged in

   Input: SSO confirmation of an email that does not match an active account in system

   Output: Session not logged in, error message shown to user

   Test Case Derivation:

   - System performs parametrized query on the Users database looking for user accounts that match the inputted confirmation's email
   - System finds no active accounts that fit the query
   - System displays error message to user, session remains not logged in

   How test will be performed: Through MsTest within the .NET solution used for implementation

5.2.5 Materials Testing

This module stores the results of materials tests manually done by club members to test the strength of a given material. Any authorized user shall be able to submit a test result and it shall be presumed to be reliable. However an executive user may mark the test unreliable, which will exclude it from all future materials analysis calculations as long as it marked as unreliable.

Automatic tests are a poor fit here because the bulk of the work is to convert a user request into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Results Submission, Group

   Type: Manual, Dynamic

   Initial State: User on materials testing page and may submit a materials test

   Input: An spreadsheet-style document (.csv or .xslx) containing materials testing results (each containing a material and its strength)

   Output: The results are stored as reliable materials testing results. User recieves confirmation of sucessful submission.

   Test Case Derivation:

   - User clicks "Add Materials Test Result" button
   - Graphical user interface (GUI) opens, prompting the upload the spreadsheet-style document
   - User uploads the document and submits it
   - System normalizes the submitted data to obtain values used to add to the materials testing database
   - System adds new rows to the database containg the obtained values and the reliable flag set to true
   - System recieves confiramation from the database that the row was added successfully
   - User recieves confirmation from the system the materials test was submitted successfully

How test will be performed: Manually, through the front end.

2. Test Marked as Unreliable

   Type: Manual

   Initial State: User on materials testing page and may deem a test unreliable

   Input: The test a user deems unreliable

   Output: The test is marked unreliable; user recieves corresponding confirmation

   Test Case Derivation:

   - User clicks "Mark Test Unreliable" button
   - System displays list of materials tests submitted, in an order descending by date.
   - User picks the test they have judged to be unreliable from the menu.
   - The user clicks the "Mark Unreliable" button.
   - A confirm dialog opens, and the user selects the "Yes, I am sure" button. The dialog closes.
   - The system submits a query to the materials testing database to mark the picked test unreliable.
   - The database marks that test unreliable and sends a confirmation to the system.
   - The system tells the user through an on-screen message that the test (with a brief description of it) was successfully marked unreliable.

   How test will be performed: Manually, through the front-end

5.2.6 Knowledge Repository

This module contains the club-wide document share and lets users add documents to it, alter or delete their own documents or any document as an

executive, and search, preview and download documents. There is a role-based access system based on account classification, so not all users may see every document added to the document share.

Automatic tests are a poor fit here because the bulk of the work is to convert a user request into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Alter Document

   Type: Manual, Dynamic

   Initial State: User is logged in and on the knowledge repository page. They also may perform their desired action.

   Input: The document name, the action, and the contents if the action is not delete.

   Output: The action is done to the document with the inputted name, adding the contents to the document if the action was not delete. Test Case Derivation:

   - User clicks one of the following buttons: (a) "Add Document", (b) "Replace Document", (c) "Edit Document" or (d) "Delete Document.
   - If the button clicked is (a) or (b):
     - A dialog opens, with an empty markdown textbox and a button to open an upload wizard.
     - The user uploads a document from the upload wizard.
     - Alternatively, the user adds to the markdown textbox.
     - The user clicks the corresponding submit button, submitting their desired input to the system.
   - If the button clicked is (c):
     - A dialog opens with a markdown textbox containing the existing markdown document.
     - The user edits the document to their heart's desire.
     - The user submits the document.

- If the user clicks button (d), the request is submitted directly.
- The system request the drive fulfill the request. The drive does and confirms it to the system.
- The system displays a confirmation message to the user.

How test will be performed: Manually, through the front end.

2. Obtain Document

   Type: Manual

   Initial State: User on knowledge repository page and access a document

   Input: The document name

   Output: The user gains access to document, with a preview (unlessit cannot be done) and a download option

   Test Case Derivation:

   - User clicks "Search for Document" button
   - User enters suitable query to search bar.
   - User hits "Search" and results are returned in declining likeness to the query.
   - The user selects their desired document.
   - A new page opens with the preview. The user is also presented with a button they may click if they wish to download the document.

   How test will be performed: Manually, through the front-end

5.2.7 Knowledge Transfer

This module tracks whether a user may leave the club soon. If so, the module will give them an alert on their user homepage, prompting them to fill out their transition document. The module then takes input through a form. Once the user submits it, the system adds it to the knowledge repository.

Automatic tests are a poor fit here because the bulk of the work is to convert user-entered text into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Submit Transfer Document

   Type: Manual, Dynamic

   Initial State: User is logged in and may leave the club soon

   Input: The text entered into each input field of the tranfer form

   Output: The text is converted into one easy-to-read document, which is added to the knowledge repository.

   - User finds alert "Make Transition Document" on user homepage.
   - User clicks the alert, opening up the transition document form
   - The user fills out the form and submits it
   - The system converts it into an easily readable and understandable document
   - The system uploads the document to the knowledge repository.
   - Once recieving a confirmation of success, it displays a "form successfully submitted" message to the user

   How test will be performed: Manually, through the front end.

5.2.8 Toolbox Broker

This module fashions and displays the homepage. It furthermore transfers the user from the home page to the pages for every other module they can access, besides the public logged out module.

Automatic tests are a poor fit here because the bulk of the work is to convert a user request into a format the database uses, not re-confirming the abilities of a highly reliable industry-standard database. Thus, manual tests shall be used.

1. Go to Service

   Type: Manual, Dynamic

   Initial State: User is logged in and is on home page. They may access the page they desire to access.

   Input: The user clicks the buttton corresponding to their desired action.

   Output: The system displays the page for their desired action.

- The user clicks the button
- The system opens up the corresponding page

How test will be performed: Manually, through the front end.

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

Most information to satisfy the non-functional requirements come from the functional tests above. Timing requirements shall be handled by time-keeping and recording a fail if the test goes overtime, even if it the desired result is ultimately obtained. Logging will be a byproduct of the functional tests, and some requirements, like weekly backup, cannot be effectively unit tested. One additional test is required, and it is detailed below.

5.3.1 Module Analysis

Unlike the counterpart subsections in the functional unit section, this is tested as one unit because the user will evaluate all its submodules in one action.

1. Timing

   Type: Manual, Dynamic

   Initial State: User on model analysis page

   Input/Condition: The STEP file of a bridge model.

   Output/Result: Whether the bridge will fail, whether the bridge is rules compliant and its score.

   How test will be performed: Manual, Dynamic

5.4 Traceability Between Test Cases and Modules

See the subsection foreword above. The crown jewel of this service, model analysis, is throughly tested; upload through the sole non-functional unit test, the three calculation sub-mdoules are formalized and tested through their formalizations, while the external CAD software library's accuracy is assumed. For each auxiliary module listed above, which contains every service this system calls for, there is at least one functional unit test case.

References

Author Author. System requirements specification. https://github.com/..., 2019.

Pedro Azevedo, Mason Azzopardi, Michael Roberts, and Jack Easton. Hazard Analysis, Troitsky Toolbox. Team 14, SFWRENG 4G06 Capstone, McMaster University, 2026a. URL https://github.com/JackEaston4/Troitsky-Toolbox/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf.

Pedro Azevedo, Mason Azzopardi, Michael Roberts, and Jack Easton. Software Requirements Specification, Troitsky Toolbox. Team 14, SFWRENG 4G06 Capstone, McMaster University, 2026b. URL https://github.com/JackEaston4/Troitsky-Toolbox/blob/main/pdfs/SRS/SRS.pdf.

Jakob Nielsen. *Usability Engineering*. Academic Press, Boston, MA, USA, 1993.

David Temoshok, James L. Fenton, Yee-Yin Choong, Naomi Lefkovitz, Andrew Regenscheid, Ryan Galluzzo, and Justin P. Richer. Digital identity guidelines: Authentication and authenticator management. NIST Special Publication 800-63B-4, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2025.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. Constants already defined in Appendix A.1 of the SRS (Azevedo et al., 2026b) keep the same name and value. Table 2 lists the constants used by the system tests for functional requirements, Table 3 the test data they use and its expected results, and Table 4 the models used to test rule compliance.

Table 2: Symbolic constants used by the system tests for functional requirements

Constant	Value	Rationale (test cases)
PASSWORD_MIN_LENGTH	15 characters	Minimum length of a single-factor password in NIST SP 800-63B-4 (Temoshok et al., 2025), required by SR-AC-2 (T.FR.AC.2.1).
PASSWORD_MAX_LENGTH	64 characters	NIST SP 800-63B-4 recommends permitting passwords of at least 64 characters, so long passwords must not be rejected or truncated (T.FR.AC.2.1).
PASSWORD_BLOCKLIST	List of commonly used, expected, or compromised passwords	Required by NIST SP 800-63B-4. The test uses an entry of at least PASSWORD_MIN_LENGTH characters, so that it can only be rejected for being on the list (T.FR.AC.2.1).
COMMON_FILE_TYPES	PDF, DOCX, XLSX, CSV, PPTX, PNG, JPG, STEP	The types of club material in the rationale of FR-KR-2: documents, spreadsheets, slides, photos of joints, and CAD models (T.FR.KR.4.1).

Constant	Value	Rationale (test cases)
PREVIEW_FILE_TYPES	PDF, PNG, JPG	Displayed natively by every browser of NFR-SYS-1, so FR-KR-4 needs no conversion (T.FR.KR.4.1).
CATEGORY_LIST	Design, Materials, Construction, Competition, Reports, Administration	The standardized category list of SR-KR-4, covering the content types of FR-KR-1. To be confirmed with the Troitsky club (T.FR.KR.4.1).
BROKER_TIMEOUT	10 s	The limit for keeping a user's attention on a task (Nielsen, 1993, ch. 5), as used for the time constants of SRS A.1. After it, the broker reports the service as unavailable (FR-BR-4) (T.FR.BR.3.1).
MA_TIMEOUT	120 s	Time limit of SR-MA-3. Four times MAX_MODEL_LOAD_TIME (30 s, SRS A.1), so that a typical club model never reaches it (T.FR.MA.5.1).
MA_TIMEOUT_TEST	1 s	Value of MA_TIMEOUT in the test deployment, so that the timeout can be tested with REF_MODEL instead of a very large model (T.FR.MA.5.1).
CALC_TOLERANCE	1% relative error	REF_MODEL is statically determinate, so a linear-static analysis should match the hand calculation apart from rounding. Unlike FAILURE_LOAD_TOLERANCE (25%, SRS A.1), it does not compare estimates with real bridges (T.FR.MA.5.1, T.FR.PA.6.1).
DIMENSION_TOLERANCE	0.5 mm	Defined in SRS A.1 for NFR-MA-3 (T.FR.MA.5.1).

Table 3: Test data for the system tests for functional requirements

Item	**Definition**	**Expected values**
REF_MODEL	STEP model of a king post truss with members of 2 mm × 10 mm cross-section ($A = 20$ mm^2): a bottom chord of two 200 mm members, two 250 mm diagonals to an apex 150 mm above mid-span, and a 150 mm vertical post. Pinned at one end, on a roller at the other, and loaded at the bottom node at mid-span.	Length 400 mm, height 150 mm, width 10 mm, total member length 1050 mm, volume 21 000 mm^3.
MAT_SET_A	Current year: five tension specimens of 2 mm × 10 mm cross-section, grain parallel to the load, failing at 760, 800, 820, 780, and 840 N. Density 0.50 g/cm^3.	Strengths 38, 40, 41, 39, 42 MPa; mean 40.0 MPa; sample standard deviation 1.58 MPa. With the 840 N specimen flagged: mean 39.5 MPa.
MAT_SET_B	Previous year: five specimens as above, failing at 680, 720, 760, 700, and 740 N. Density 0.50 g/cm^3.	Strengths 34, 36, 38, 35, 37 MPa; mean 36.0 MPa; sample standard deviation 1.58 MPa.
Analysis of REF_MODEL	The post carries the full load P in tension (the diagonals carry $0.83P$ and the chord $0.67P$), so by SRS S.7.2 the post fails first, when $P/A = \mu_{strength}$, giving $P_{fail} = \mu_{strength} \cdot A$. Mass is the volume times the density: 21 cm^3 × 0.50 g/cm^3.	Mass 10.5 g. With MAT_SET_A: failure load 800 N, load-to-mass ratio 76.2 N/g. With MAT_SET_B: 720 N, 68.6 N/g.

Item	Definition	Expected values
MT_IMPORT_FILE	CSV file in the standardized template of NFR-MT-1, containing specimens 2 to 5 of MAT_SET_A and four invalid rows: a missing applied load, a thickness of -2 mm, an applied load of -50 N, and a width of 0 mm.	Specimens 2 to 5 stored; the four invalid rows rejected.
RULESET_TEST	Rule set for a test competition year, with values chosen for testing only: mass $\leq$ 750 g; length from 900 to 1100 mm; width from 100 to 200 mm; clearance height $\geq$ 150 mm; and a materials rule (popsicle sticks, toothpicks, white glue, and dental floss only) that cannot be checked from a model.	See Table 4.

Table 4: Compliance models for RULESET_TEST. Their geometry is exact, so the extracted values equal those listed.

Model	Mass (g)	Length (mm)	Width (mm)	Clearance (mm)	Expected result
RC_PASS	500	1000	150	200	Passes every rule
RC_LIMIT	750	1100	100	150	Passes every rule (limits are inclusive)
RC_HEAVY	760	1000	150	200	Fails mass only
RC_LONG	500	1110	150	200	Fails length only
RC_SHORT	500	890	150	200	Fails length only
RC_WIDE	500	1000	210	200	Fails width only
RC_NARROW	500	1000	90	200	Fails width only
RC_LOW	500	1000	150	140	Fails clearance only

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Mason Azzopardi

1. Group communication and work delegation; soon after the SRS deliverable we were able to get int a meeting and go through the rubrics for this submission and split up the work. All group members were active in the team chat and were on time for every meeting they attended. Also throughout defining the System NFR Tests, I made sure to keep track of every NFR and SR that I was basing them off of so creating the traceability matrix went much easier than similar matrices have gone in the past
2. The main pain points for me were framing the tests in a concrete manner, I had a general Idea of what I wanted them to test and how I wanted that to be accomplished, but to actually put that to paper in a concrete and understandable manner was harder than expected.
3. I need a much stronger understanding of unit tests and how to implement them, I have very little background and will need to do a large review on .NET testing frameworks for C#.

4. I could read through .NET documentation or go through a online course like this one. I will probably take a online course as I learn better through practice and interaction rather than just reading documentation.

Pedro Azevedo

1. In writing this deliverable, the planning for the interviews and usability testing went well. I believe this was primarily due to the extensive work the team has already done in previous documentation, such as the SRS, outlining the system requirements, including usability requirements and usage scenarios. This allowed for tests to be clearly defined and directly linked to the requirements they were meant to test.

2. The main pain point during this deliverable came from the difficulty in determining the depth of interviews and usability testing questions. While potentially beneficial, it was important to keep questions relatively basic and non-complex, ensuring that interviewees would not become overwhelmed or confused. This was resolved by focusing on the core features the questions are looking to discover, removing and mitigating any more high level or technically dense questions, for the sake of user feasibility and understanding.

3. A skill that I would like to acquire to improve the success of the team in verification and validation is browser end-to-end testing, potentially with Playwright. This would help test the browser behaviour of services and of the system as a whole, ensuring that core system features and performance are functioning as development occurs.

4. To acquire and improve on this skill, I will begin with the documentation provided by Microsoft for Playwright, supplementing with youtube videos regarding the topics. I will also work with Playwright from the beginning, integrating it into the development of the POC. These together wil give me both technical and practical understanding of the tool.

Jack Easton

1. During the writing of this deliverable. I feel our team was able to find our flow together more easily. Since we have multiple deliverables under our belt at this point, we had a very solid shared team understanding. This allowed for a straightforward collaboration process where each member clearly could understand what each part of the system needed to do and how it needed to be tested.

2. For this deliverable, it was particularly difficult to approach the system tests of the functional and non-functional requirements. This is just due to the large number identified in the SRS and Hazard Analysis. Ensuring all requirements were covered and appropriately assessed was quite time consuming. However, our team came goether well to ensure correctness and that all of these requirements were covered.

3. I think personally I need to become more farmiliar with specific testing tools used in industry that we could adopt in this project. I do not have very much complex experience with dedicated static checkers, linters, or dynamic testing tools. I think that learning how to use these well would be invaluable in the verification and validation of this project.

4. For me, I plan to dive into researching tools and standards that are used in industry and figure out how to apply and use them in our project. I think also hands-on tinkering with these tools would provide the best learning for me and give me the confidence to use these tools well.